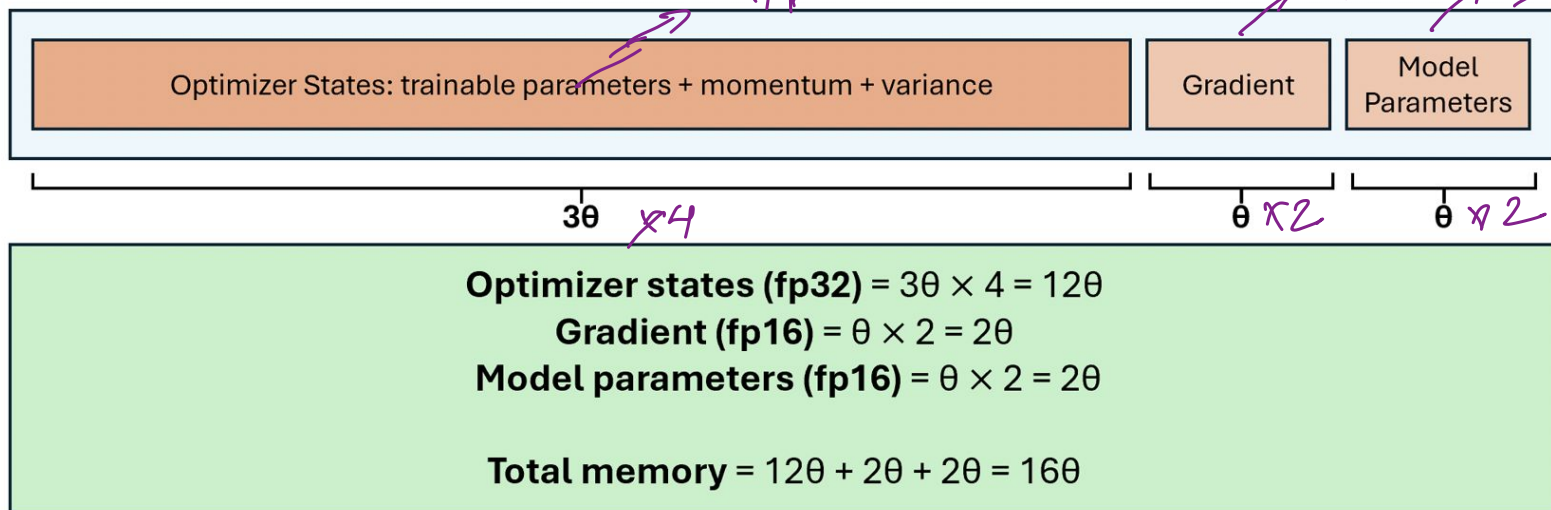


Efficient Fine-tuning for Quantized LMs: QLoRA

CS60216: Safety Fundamentals for Generative AI

Feb 5th, 2026

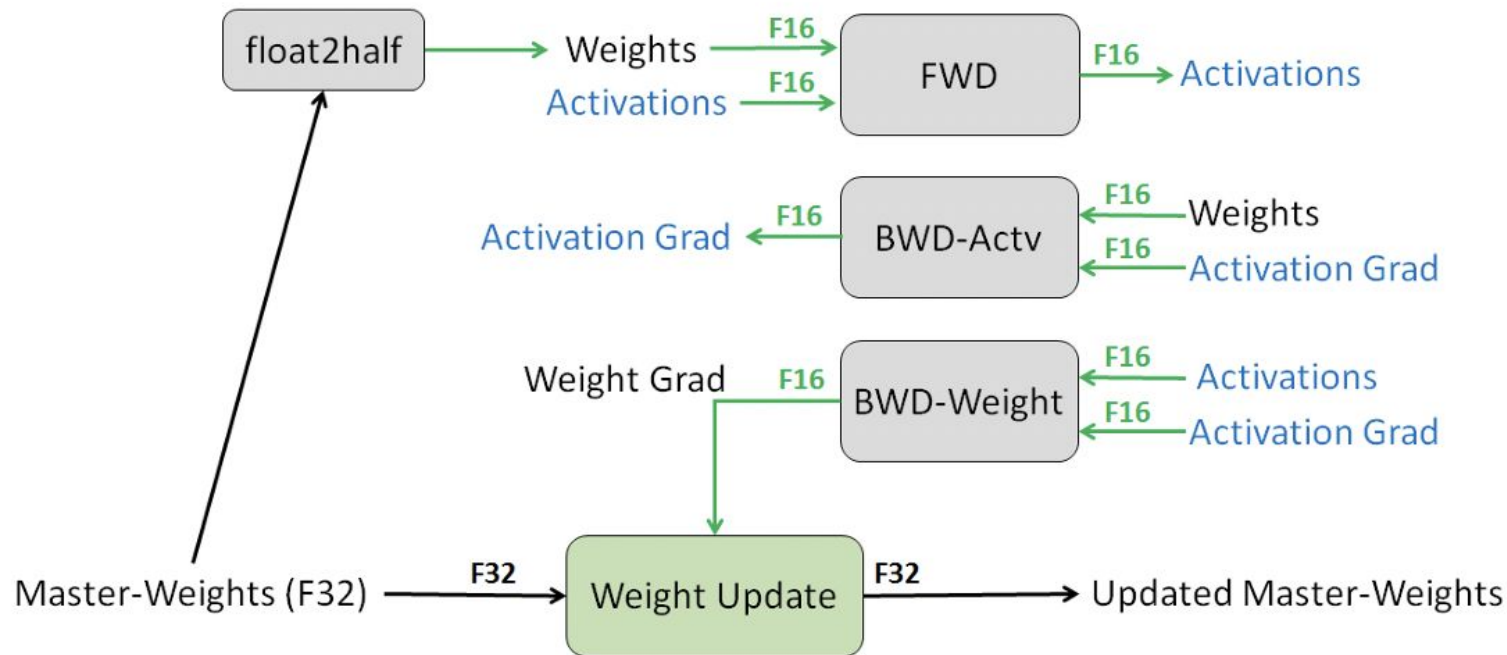
GPU Memory Requirements: Model state



Note: memory required can vary with precision and optimizer

Model state memory for a model with θ parameters, when fine-tuned using Adam optimizer under ***mixed-precision setting***. Model state comprises of optimizer state, gradients and model parameters. Total model state memory, with no optimization enabled, adds up to 16θ bytes.

Mixed-precision setting



Need for master copy? updates (weight gradients multiplied by the learning rate) become too small to be represented in FP16.

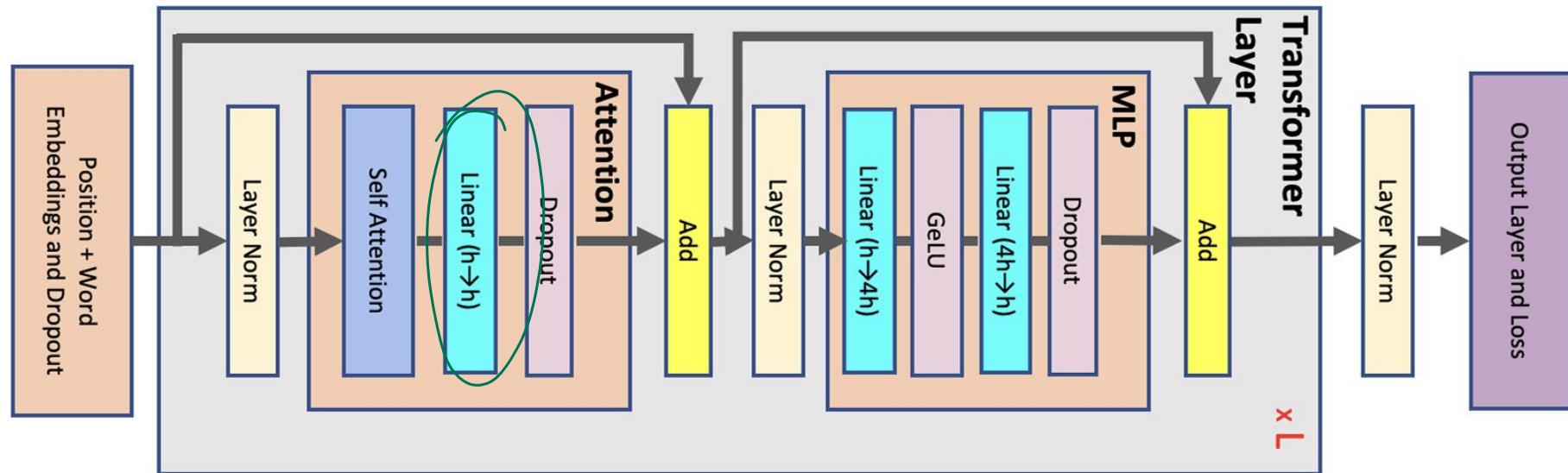
Seq length s

batch size $\rightarrow b$

model dim $\rightarrow d$

Sp'16

Activation Memory



Activations are stored in a 16-bit floating point format and therefore each element requires 2 bytes for storage. The only exceptions are the dropout masks which only require a single byte per element.

Activation memory

Attention block: which includes self attention followed by a linear projection and an attention dropout. The linear projection stores its input activations with size $2sbh$ and the attention dropout requires a mask with size sbh .

(instead of d)

Not in the syllabus

- **Query (Q), Key (K), and Value (V) matrix multiplies:** We only need to store their shared input with size $2sbh$.
- **QK^T matrix multiply:** It requires storage of both Q and K with total size $4sbh$.
- **Softmax:** Softmax output with size $2as^2b$ is required for back-propagation.
- **Softmax dropout:** Only a mask with size as^2b is needed.
- **Attention over Values (V):** We need to store the dropout output ($2as^2b$) and the Values ($2sbh$) and therefore need $2as^2b + 2sbh$ of storage.

Summing the above values, in total, the attention block requires $11sbh + 5as^2b$ bytes of storage.

s : sequence length, b : batch size

h : model dimension, a : number of attention heads

Activation memory

MLP: The two linear layers store their inputs with size $2sbh$ and $8sbh$. The GeLU non-linearity also needs its input with size $8sbh$ for back-propagation. Finally, dropout stores its mask with size sbh . In total, MLP block requires $19sbh$ bytes of storage.

Layer norm: Each layer norm stores its input with size $2sbh$ and therefore in total, we will need $4sbh$ of storage.

Summing the memory required for attention, MLP, and the layer-norms, the memory required to store the activations for a single layer of a transformer network is:

$$\text{Activations memory per layer} = sbh \left(34 + 5 \frac{as}{h} \right). \quad (1)$$

Handwritten note: $\approx 116B$

Typical scenario (Llama-2 7 B): $s=256, b=8, h=4096, L=32, a=32$

s: sequence length, b: batch size

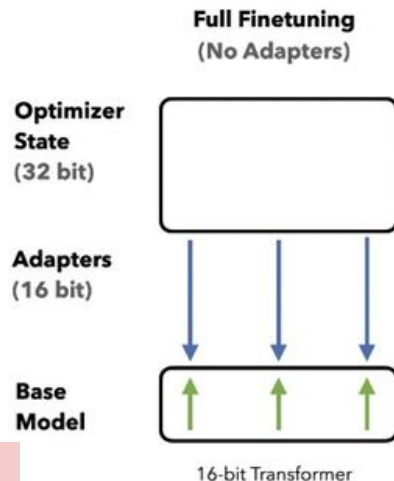
h: model dimension, a: number of attention heads

Fine-tuning is expensive

Finetuning cost per parameter:

- Weight: 16 bit
- Weight gradient: 16 bit
- Optimizer state: 64 bit
- 12 byte per parameter

*more conservative estimate, e.g.,
for SGD with momentum*

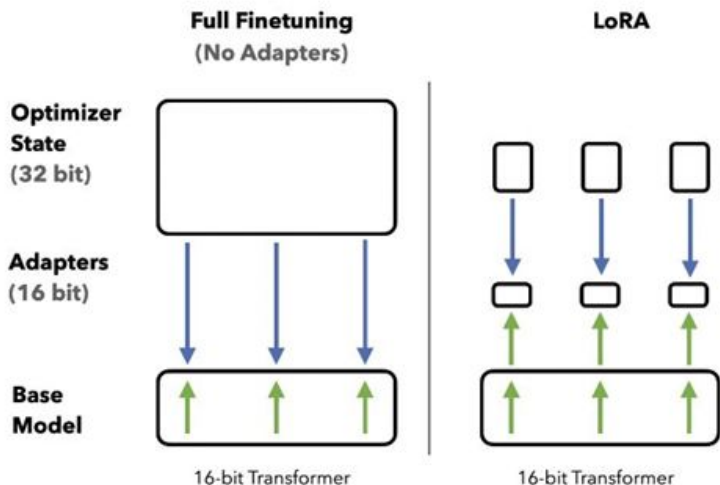


70B model -> 840 GB of GPU memory -> 36x consumer GPUs

Fine-tuning with LoRA

Finetuning cost per parameter:

- Weight: 16 bits
- Weight gradient: $\sim 0.4\text{bit}$
- Optimizer state: $\sim 0.8\text{bit}$
- Adapter weights: $\sim 0.4\text{bit}$
- 17.6 bits per parameter



70B model -> 154 GB of GPU memory -> 8x consumer GPUs

QLoRA: 4-bit frozen base model + LoRA

Finetuning cost per parameter:

- Weight: 4 bit
- Weight gradient: ~0.4 bit
- Optimizer state: ~0.8 bit
- Adapter weights: ~0.4 bit
- 5.2 bit per parameter

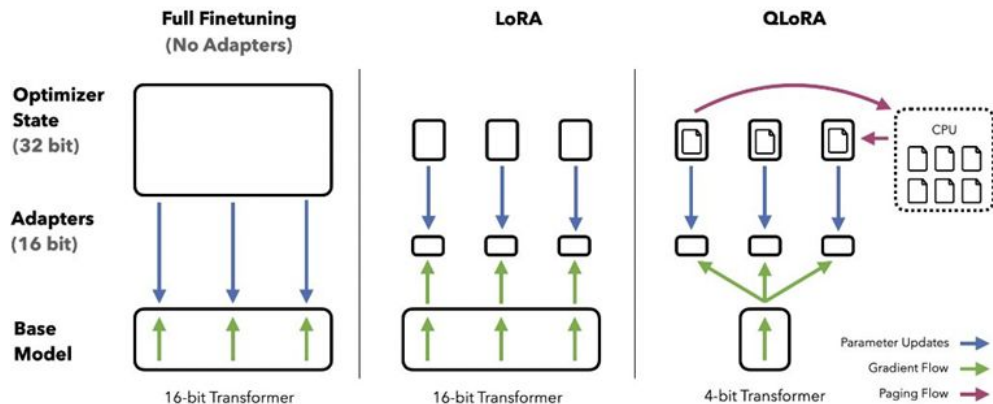


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

70B model -> 46 GB of GPU memory -> 2x consumer GPUs.

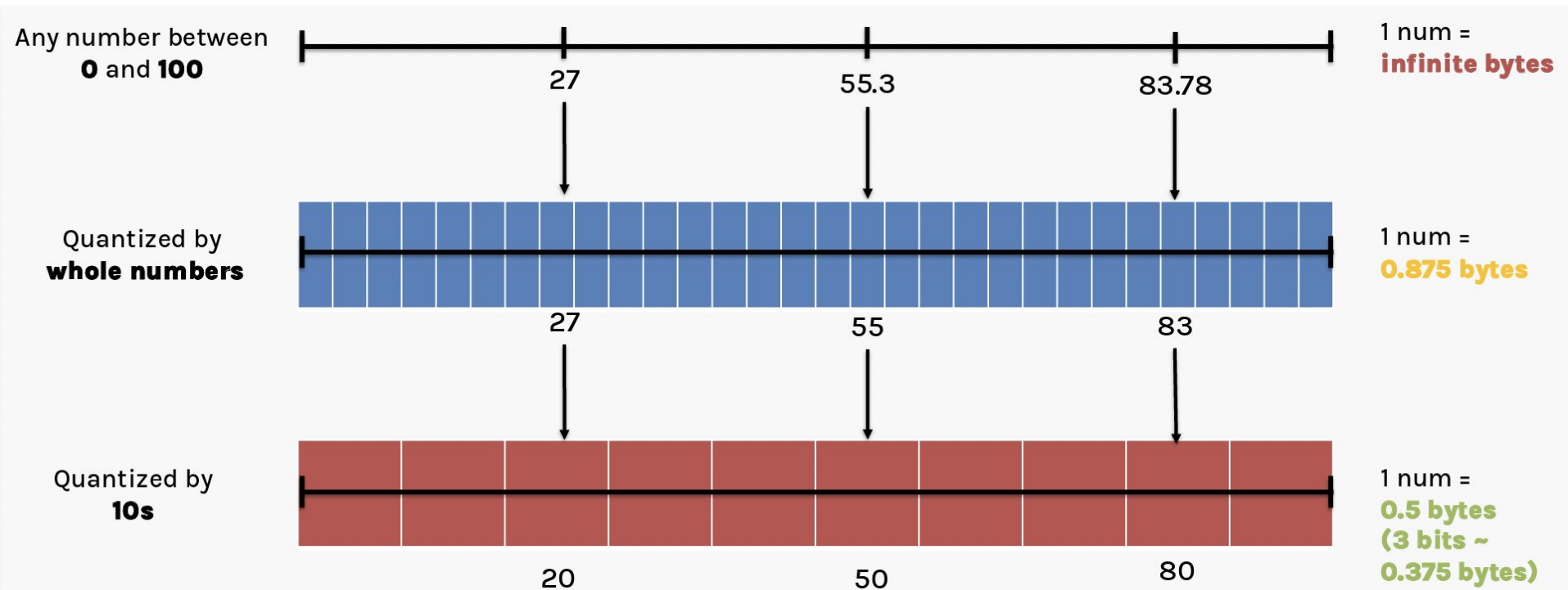
Model	Fine-tuning memory
T5-11B	132 GB
Mistral-7B	84 GB
LLaMA2-70B	840 GB

*Conservative estimate
of 12*parameters*



Model	Fine-tuning memory
T5-11B	6 GB
Mistral-7B	5 GB
LLaMA2-70B	46 GB

What is quantization?



Quantization example

$$10.2 \times 10.2$$

$$C = \left(\frac{127}{10.2} \right)$$

Let's look at an example!

Let X^{FP32} be an array of values.

Here, FP32 refers to a 32-bit floating-point number.

1.5	2.3	3.7	4.1	5.6	6.8	7.9	8.4	9.2	10.2
-----	-----	-----	-----	-----	-----	-----	-----	-----	------

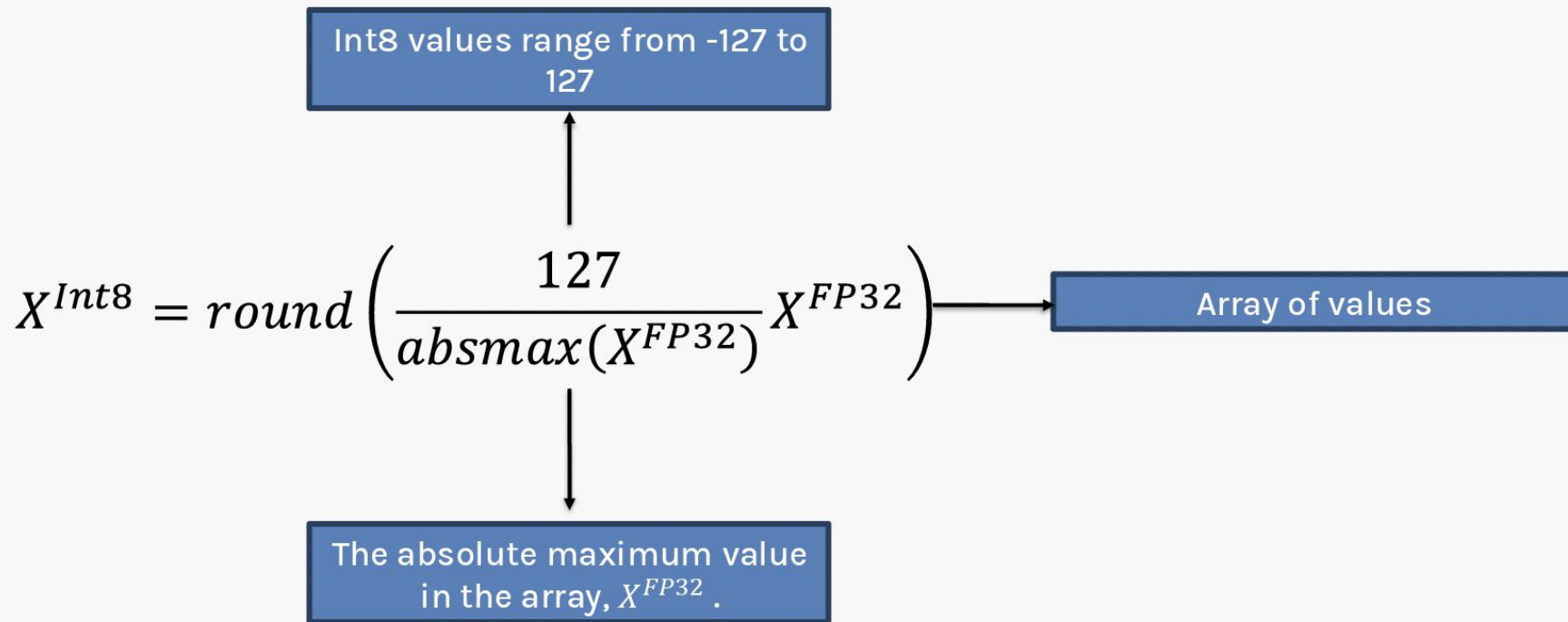
$$-5.6 \quad -4.1 \quad -3.7 \quad -2.3 \quad -1.5$$

What if we want to quantize from FP32 to Int8?

$$-127 \quad \text{---} \quad 127$$

Quantization example

So, to quantize X^{FP32} to X^{Int8} :



Quantization example

So, to quantize X^{FP32} to X^{Int8} :

$$X^{Int8} = round \left(\frac{127}{absmax(X^{FP32})} X^{FP32} \right)$$

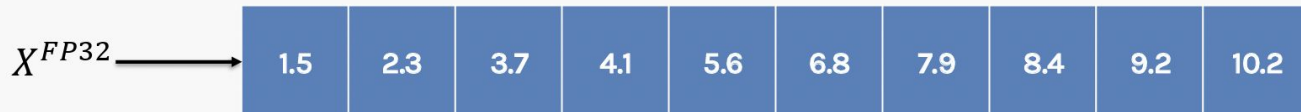


$$X^{Int8} = round(\underbrace{c^{FP32}}_{\text{scale factor}} \underbrace{X^{FP32}}_{\text{input}})$$

Quantization example

$$X^{Int8} = round(c^{FP32} X^{FP32})$$

In our example,



$$c^{FP32} = \frac{127}{absmax(X^{FP32})} = \frac{127}{10.2} = 12.4509$$



Now, we combine the formula and the values that we have

Quantization example

$$\frac{1}{12.4509}$$

$$X^{Int8} = round(\underline{12.4509} \times \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 1.5 & 2.3 & 3.7 & 4.1 & 5.6 & 6.8 & 7.9 & 8.4 & 9.2 & 10.2 \\ \hline \end{array})$$

$$X^{Int8} = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline \overset{9}{\cancel{18}} & 29 & 46 & 51 & 69 & 85 & 98 & 105 & \textcircled{115} & 127 \\ \hline \end{array}$$

$\underline{127}$ $\underline{12.4509}$ $\underline{127}$

Voila! That's how we quantize from FP32 to Int8 using the formula:

$$X^{Int8} = round(c^{FP32} X^{FP32})$$

Quantization example

$$X^{Int8} = round(c^{FP32} X^{FP32})$$

What if we want to **dequantize** and get back the original array, X^{FP32} ?

To dequantize:

$$X^{FP32} = \frac{X^{Int8}}{c^{FP32}}$$



QLoRA: Major innovations

- Major innovations:
 - 4-bit Normal Float
 - Double Quantization
 - Page Optimizer
- 4-bit storage data type
- Bfloat16 computational data type

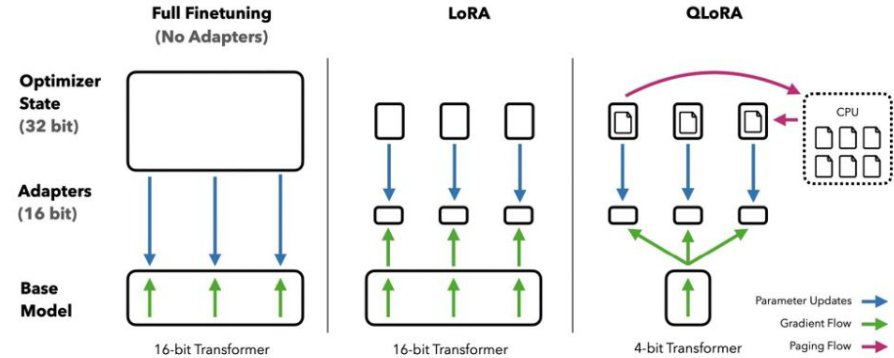


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.



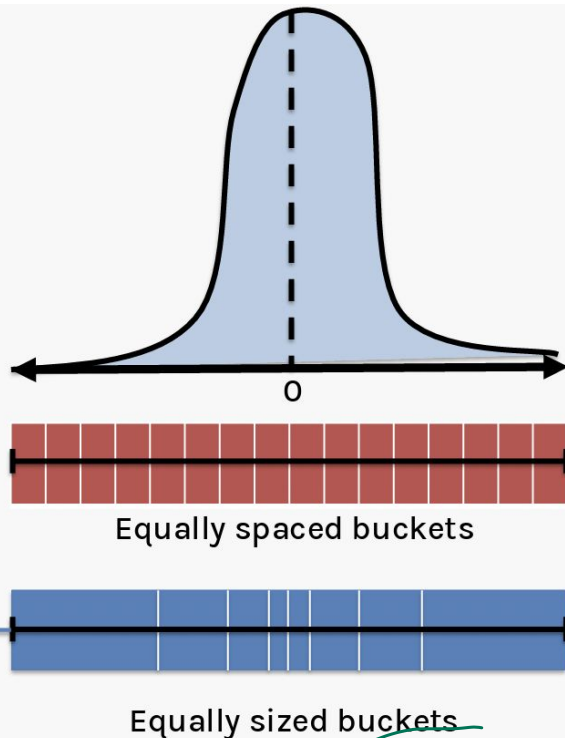
QLoRA Ingredients 1: 4-Bit NormalFloat

4-bit NormalFloat

4-bit NormalFloat is a clever way to split the buckets.

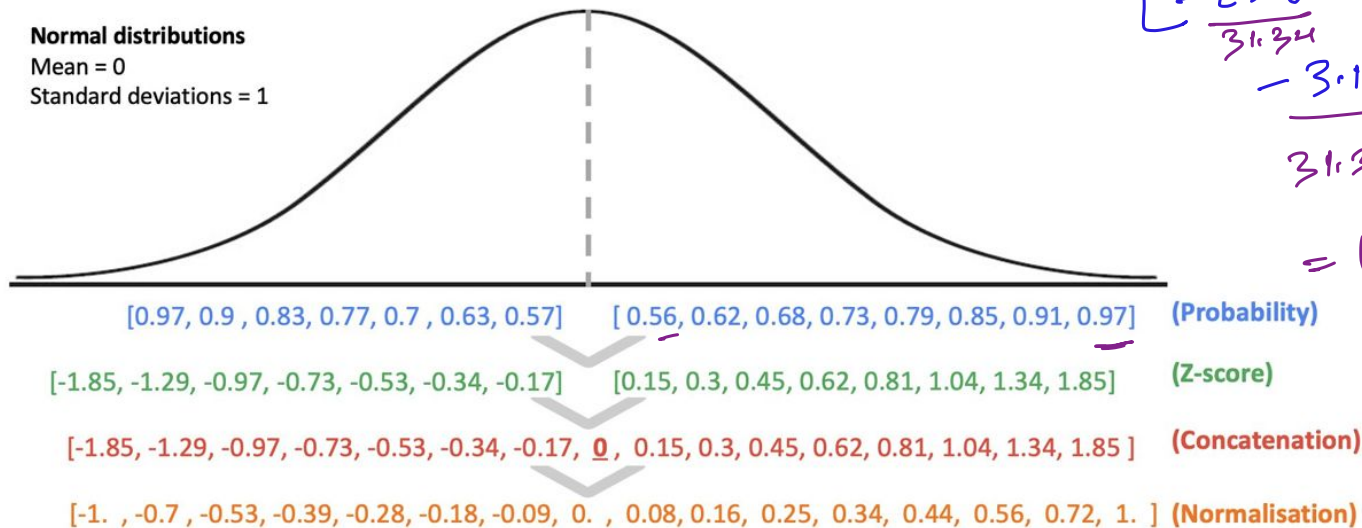
4-bit means we have $2^4 = 16$ possible buckets for quantization.

Most datasets in machine learning are normally distributed and precision around the mean is valuable.



This is an enhanced version of **quantile quantization.**

Exact values of the NF4 data type



Steps for generating the NF4 data type values:

1. Generate 8 evenly spaced values from 0.56 to 0.97 (Set I).
2. Generate 7 evenly spaced values from 0.57 to 0.97 (Set II).
3. Calculate the z-score values for the probabilities generated in Step 1 and Step 2. For Set II, calculate the negative inverse of the z-scores.
4. Concatenate Set I, a zero value, and Set II together.
5. Normalize the values by dividing them by the absolute maximum value.

<https://lmsystem.github.io>

$$\begin{bmatrix} \frac{-25.62}{31.34} & \frac{31.34}{31.34} & \frac{2.68}{31.34} \\ -3.14 & 0 & \end{bmatrix}$$

$$31.34$$

$$= \begin{bmatrix} -0.81 & 1 & 0.09 \\ -0.1 & 0 & \end{bmatrix}$$

2

$$\begin{bmatrix} -0.7 & 1 & 0.08 \\ -0.09 & 0 & \end{bmatrix}$$

$$\begin{bmatrix} 0001 & 1111 & -- \end{bmatrix}$$

QLoRA: Quantization Example for 2-bit

$\lceil 10 \rceil$ 3 3 5/3

Given a tensor X of any real data type. We can apply quantization as follows:

1. Normalize X into the range $[-1.0, 1.0]$ by dividing by $\text{absmax}(X)$
2. Find the closest value in the data type (rounding for integers; in general binary search)

Map: {Index: 0, 1, 2, 3 $\rightarrow$ Values: -1.0, 0.3, 0.5, 1.0}

We saw how to get these values for NF4

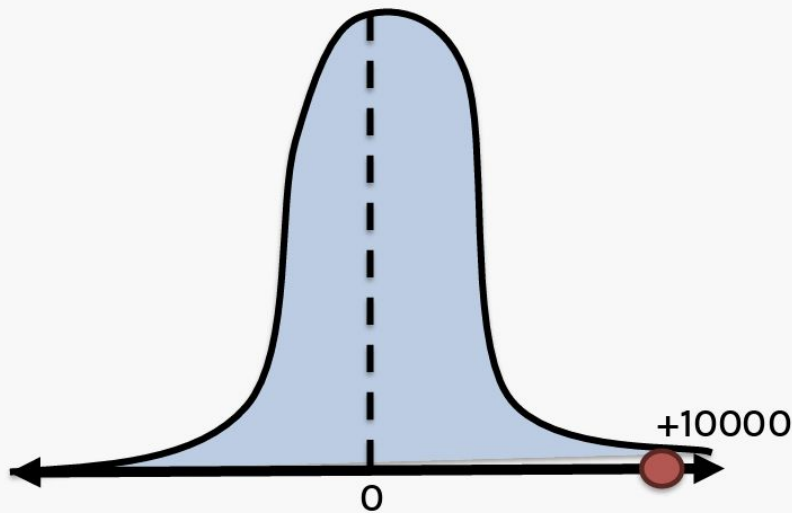
Input tensor: [10, -3, 5, 4]

1. Normalize with absmax : [10, -3, 5, 4] $\rightarrow$ [1, -0.3, 0.5, 0.4]
2. Find closest value: [1, -0.3, 0.5, 0.4] $\rightarrow$ [1.0, 0.3, 0.5, 0.5]
3. Find the associated index: [1.0, 0.3, 0.5, 0.5] $\rightarrow$ [3, 1, 2, 2] $\rightarrow$ store
4. Dequantization: load $\rightarrow$ [3, 1, 2, 2] $\rightarrow$ lookup $\rightarrow$ [1.0, 0.3, 0.5, 0.5] $\rightarrow$ denormalize $\rightarrow$ [10, 3, 5, 5]

QLoRA Ingredients 2: Double Quantization

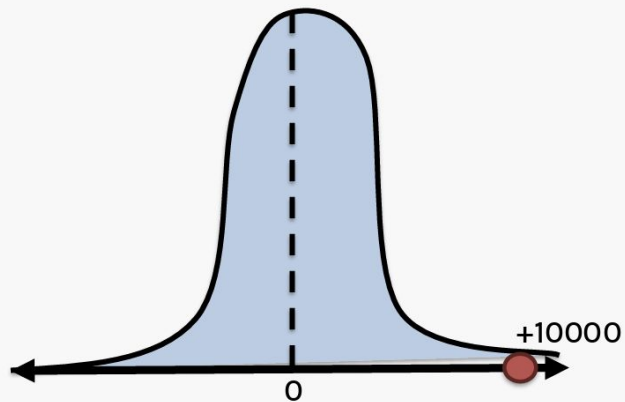
QLoRA Ingredients 2: Double Quantization

Let's see how the weight tensors look like on the graph.



This is **unbounded** and could take up any **maximum value** (an outlier!)

QLoRA Ingredients 2: Double Quantization



This is **unbounded** and could take up any **maximum value**

This could introduce **bias** in our quantization process

$$W^{Int8} = round(\frac{127}{absmax(W^{FP32})} W^{FP32})$$

So, how to avoid this problem?

QLoRA Ingredients 2: Double Quantization

The answer to that is: **Block-wise Quantization**, which is the first step in Double Quantization!

Let's look at an example to understand this concept

100 bytes

Normal
Int 8
 $\rightarrow 25 + 4 = 29$

Weight Tensor (W^{FP32})



-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1

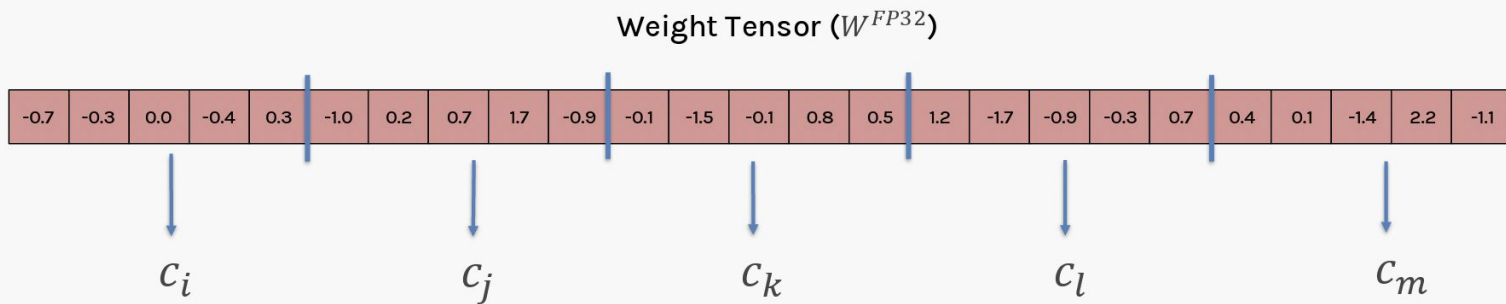
9
9
45
bytes

QLoRA Ingredients 2: Double Quantization

We flatten the matrix as follows:

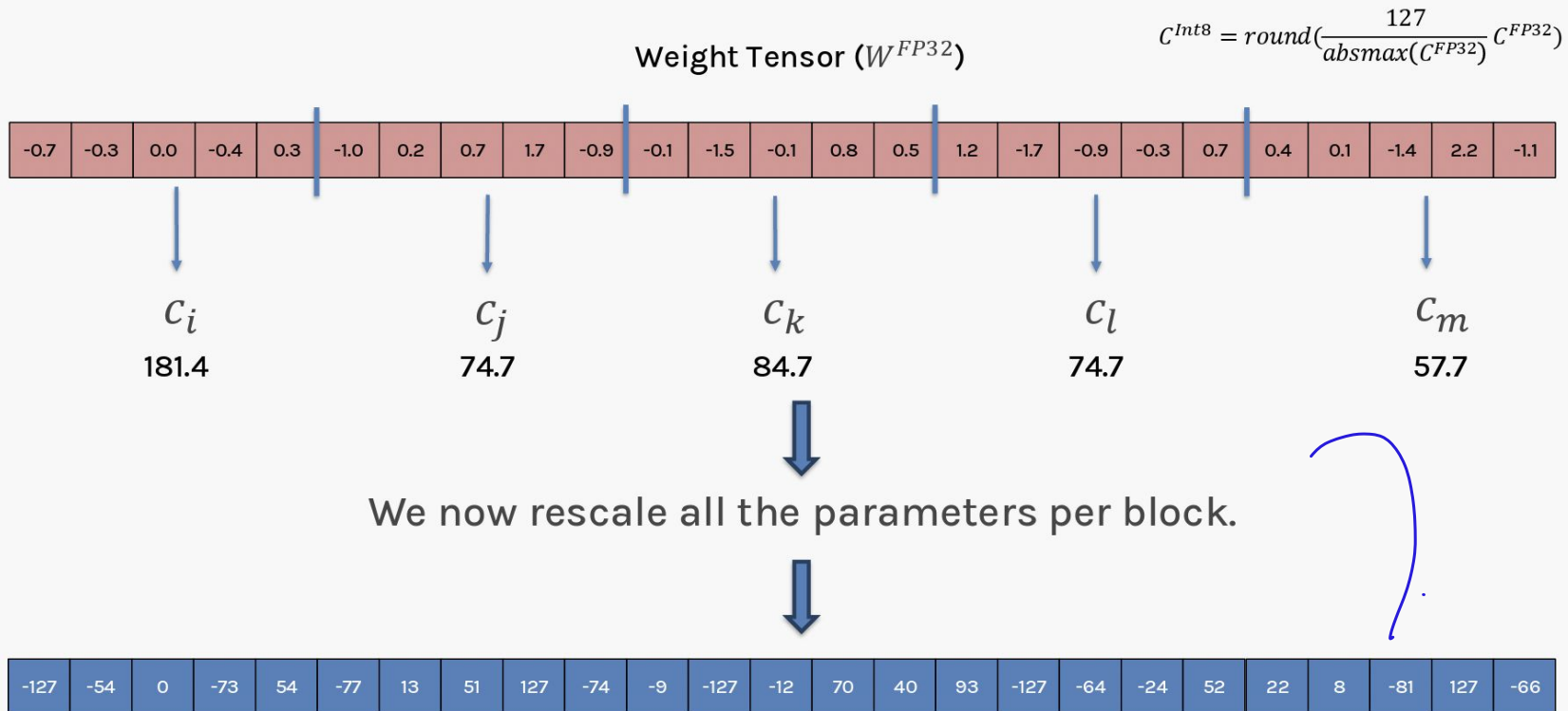
Now we divide it up into different blocks.

We calculate the **quantization constants** for each block.



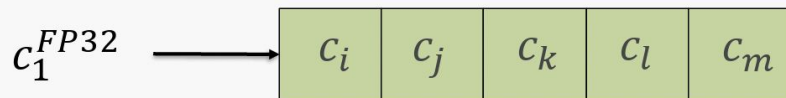
If there are any outliers in a block, they won't affect the quantisation in the other blocks.

QLoRA Ingredients 2: Double Quantization



QLoRA Ingredients 2: Double Quantization

We now have a new array:



c_1^{FP32} is an array of all the constants from each block of the Weight Tensor.

Now, we repeat the same process of quantization for the quantization constants.

$$c_1^{Int8} = \text{round}\left(\frac{127}{\text{absmax}(c_1^{FP32})} c_1^{FP32}\right)$$

$$c_1^{Int8} = \text{round}(c_2^{FP32} c_1^{FP32})$$

Double Quantization

100 bytes
↓
29 bytes
Normal
25 + 5 + 4
= 34

Let's see the difference in memory usage before and after Double Quantization

QLoRA Ingredients 2: Double Quantization

Before

All we had was a weight matrix containing FP32 values.

In our example, we had a 5x5 matrix.

Each value was 4 bytes in size.

So, the total memory used
was:

25x4=100 bytes



-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1

Weight Tensor (W^{FP32})

Let's see the difference in memory usage before and after Double Quantization

QLoRA Ingredients 2: Double Quantization

After

-127	-54	0	-73	54
-77	13	51	127	-74
-9	-127	-12	70	40
93	-127	-64	-24	52
22	8	-81	127	-66

Rescaled Weight Tensor
(W^{Int8})

25x1=25 bytes.

c_i	c_j	c_k	c_l	c_m
-------	-------	-------	-------	-------

c_1^{Int8}

5x1=5bytes.

c_2^{FP32}

4 bytes

So, in total:

25 + 5 + 4 = **34 bytes**



QLoRA Ingredients 2: Double Quantization

After

-127	-54	0	-73	54
-77	13	51	127	-74
-9	-127	-12	70	40
93	-127	-64	-24	52
22	8	-81	127	-66

c_i	c_j	c_k	c_l	c_m
-------	-------	-------	-------	-------

c_2^{FP32}

$25 + 5 + 4 = \mathbf{34 \text{ bytes}}$

Before

-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1

$25 \times 4 = 100 \text{ bytes}$

That is an approximate 70% reduction in memory usage!!

QLoRA Ingredients 3

Before we talk about the third ingredient in **QLoRA**, let's talk about a problem.

A problem which all of us have faced while training a Neural Network

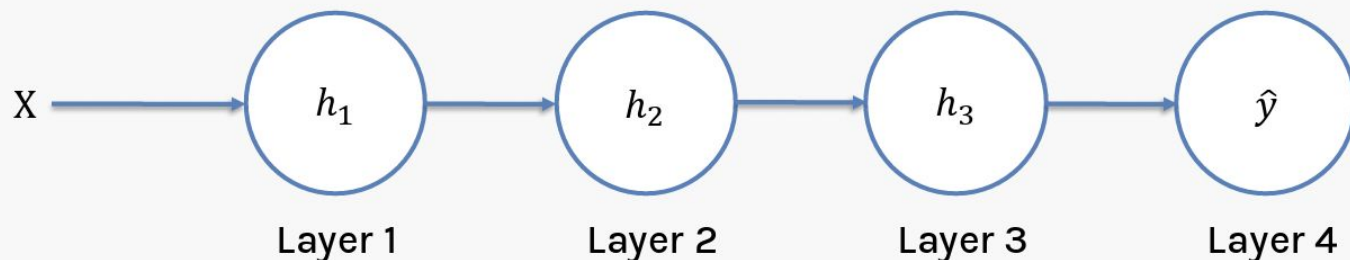
Running Out of Memory!

So, how do we train a modern Neural Networks without taking a hit on the memory?

We use **gradient checkpointing**.

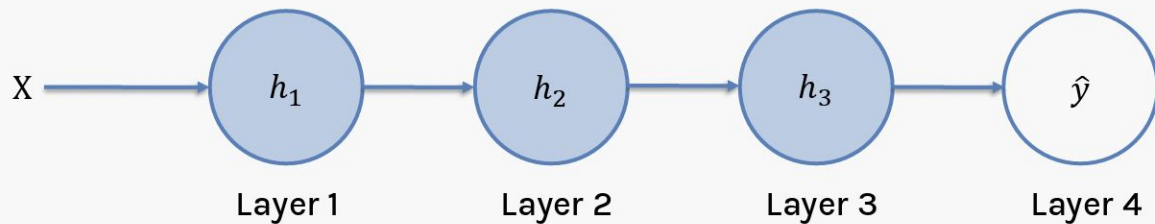
QLoRA Ingredients 3

Imagine this simple neural network



When we do a forward-pass, we calculate the activations for each layer.

QLoRA Ingredients 3



100 layers

However, this takes up precious memory.

Modern-day computers have become very efficient at **parallel processing**. What they lack is memory.

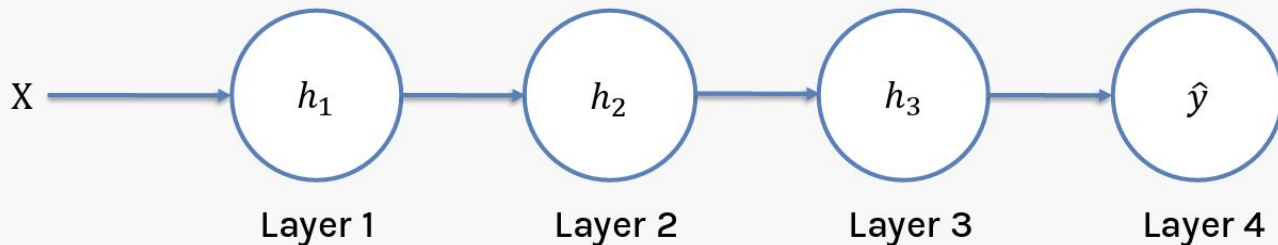
We don't need to store all the hidden states.

QLoRA Ingredients 3

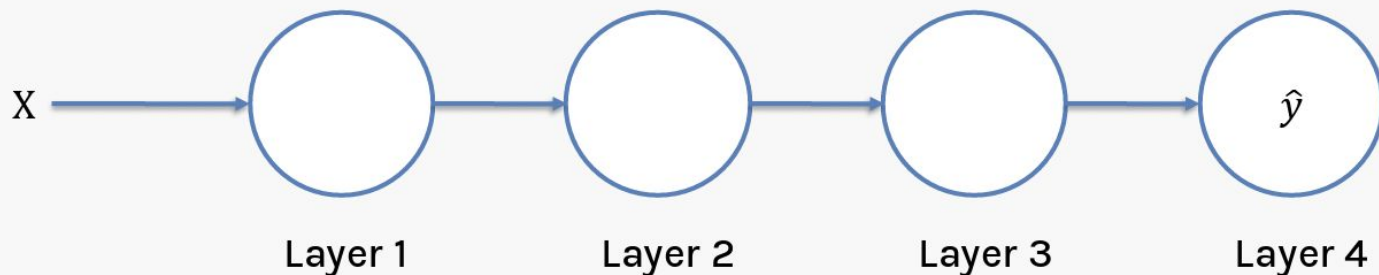
We only store in memory what is needed at the moment.

We keep **discarding** **activations** that have already been used to calculate the **next dependent hidden state's activation**.

So, let's see how it looks!



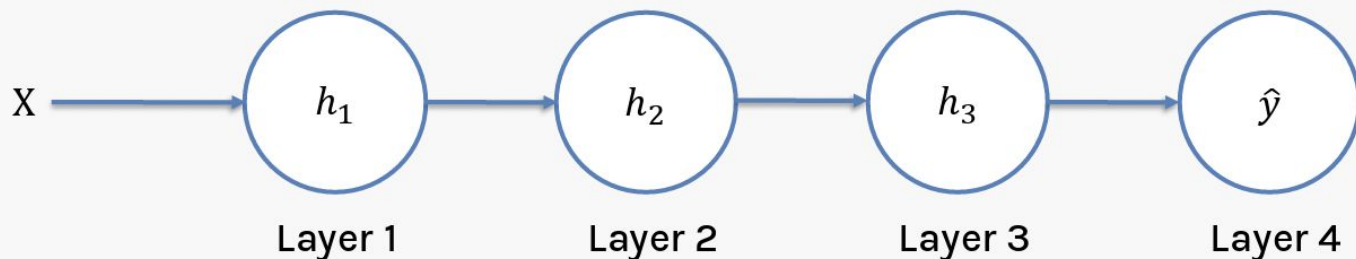
QLoRA Ingredients 3



During backpropagation, we must recompute all the discarded activations.

To manage this, we introduce checkpoints in the middle.

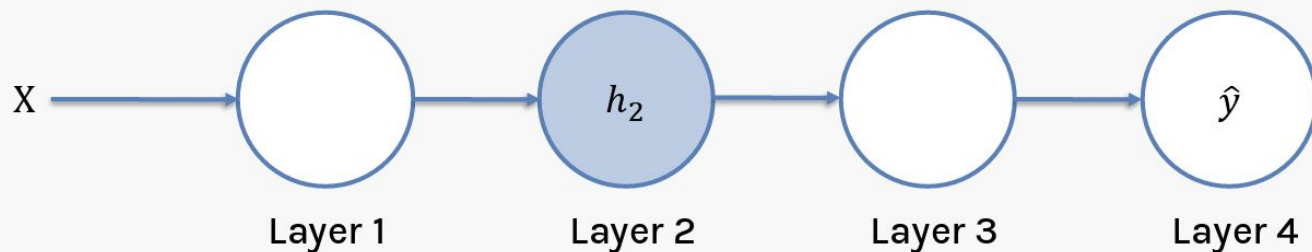
QLoRA Ingredients 3



Checkpoints are usually placed at every $\sqrt{n}$ layer, considering we have a n -layer neural network.

So, now when we re-compute the activations for **backward pass**, we don't have to start from the beginning!

QLoRA Ingredients 3



This allows us to mitigate the **OOM (Out of memory) error** to some extent, but it doesn't get rid of it!

We still see some **memory spikes** especially when we pass in long sequences in the batch.

QLoRA Ingredients 3: Paged Optimizer

Paged Optimizer - Looping in your CPU



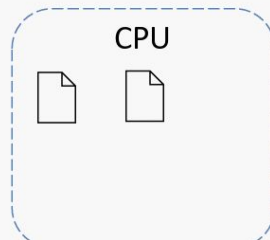
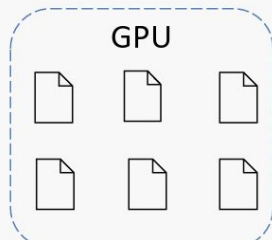
Paging is a memory management technique, where RAM is divided into fixed-size blocks called 'pages'

It does automatic page-to-page transfers between CPU and GPU

Avoids the gradient checkpointing memory spikes that occur when processing a mini batch with a long sequence length.



GPU Memory has space now.



Now that the GPU has space, when a page moved to CPU is required, we move it back to GPU for computation.

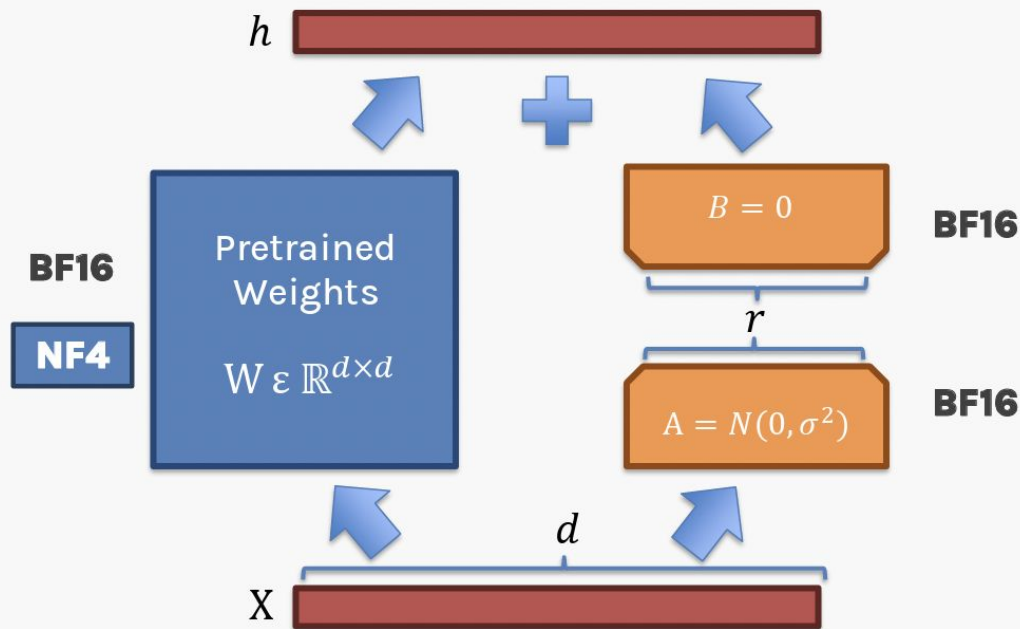
QLoRA: Putting it all together

Forward Pass

During the **forward pass**, we first dequantize the W weights from **NF4** to **BF16** for computation.

We then use the **BF16** values of W , A and B to perform the required calculations.

The **BF16** values of W is then **deleted** to save on storage!

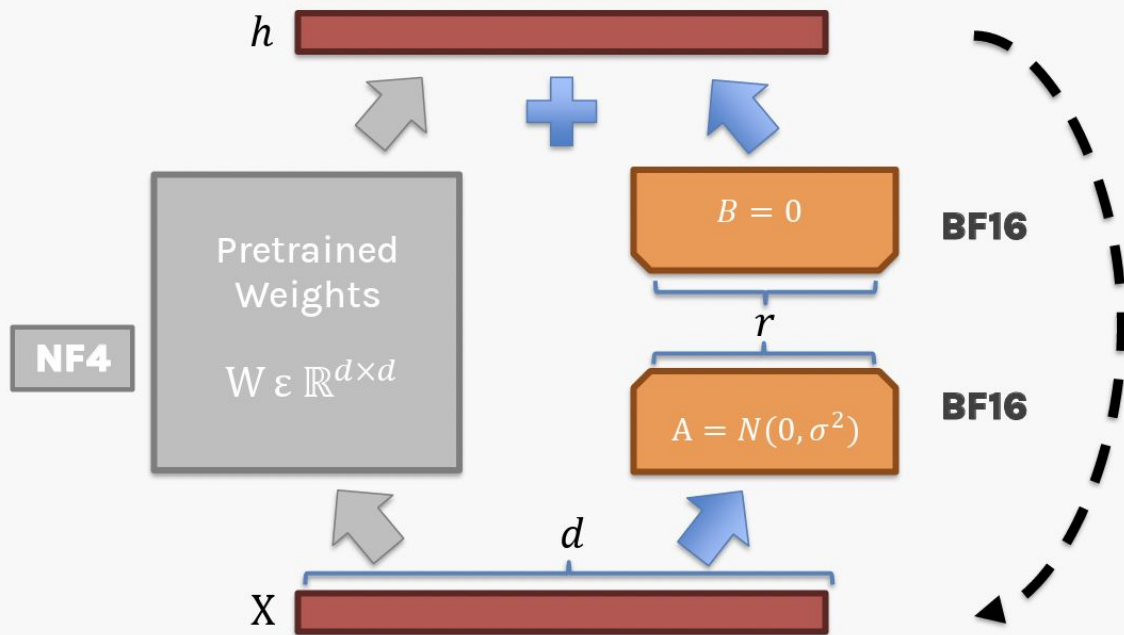


QLoRA: Putting it all together

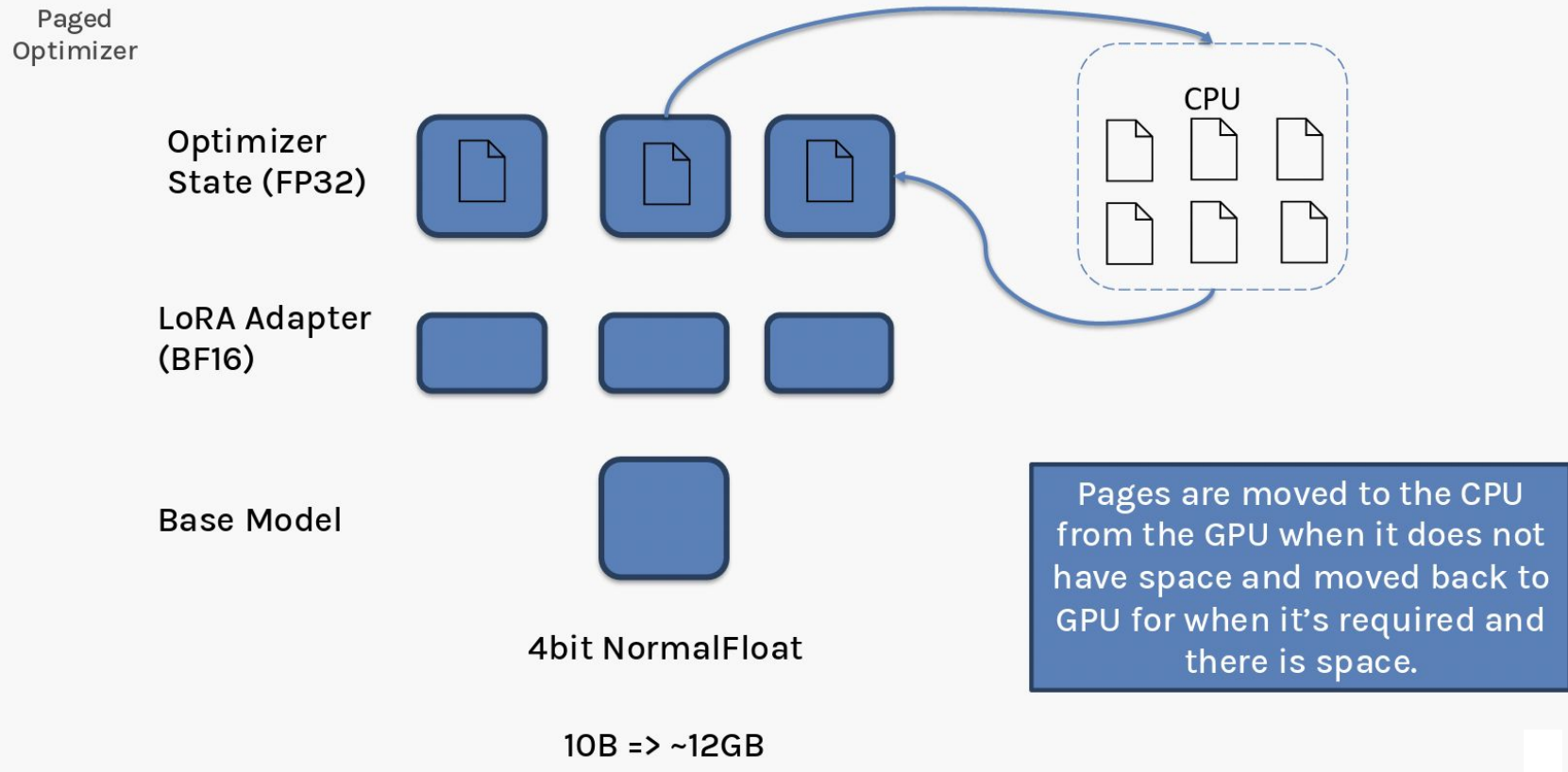
Backward Pass

As in LoRA, we keep W weights frozen and allow the **gradients** to only flow through the **adapters**.

We then repeat the cycle of forward and backward passes till a minima is reached.



QLoRA: Putting it all together



QLoRA: Putting it all together

Putting it mathematically,

Let's start with **LoRA**:

Weights of LoRA: $W_0 + \frac{\alpha}{r} BA$

The diagram shows the equation $W_0 + \frac{\alpha}{r} BA$ with four callout boxes pointing to its components: 'Initial LLM Weights' points to W_0 ; 'Scaling parameter' points to α ; 'Decomposed matrices' points to BA ; and 'Rank of B and A' points to r . The fraction $\frac{\alpha}{r}$ is circled in blue.

- Initial LLM Weights
- Scaling parameter
- Decomposed matrices
- Rank of B and A

Forward pass in LoRA: $Y = \underline{XW_0} + \frac{\alpha}{r} \underline{XBA}$

The diagram shows the equation $Y = XW_0 + \frac{\alpha}{r} XBA$ with blue underlines under XW_0 and XBA .

QLoRA: Putting it all together

$$Y = XW_0 + \frac{\alpha}{r} XBA$$

Let's expand the formula and see how it looks!

$$Y^{BF16} = X^{BF16} \text{doubleDequant}(c_1^{FP32}, c_2^{k-bit}, W_o^{NF4}) + \frac{\alpha}{r} X^{BF16} B^{BF16} A^{BF16}$$

$$\begin{aligned} \text{where } \text{doubleDequant}(c_1^{FP32}, c_2^{k-bit}, W_o^{NF4}) &= \text{dequant}(\text{dequant}(c_1^{FP32}, c_2^{k-bit}), W_o^{4bit}) \\ &= W^{BF16} \end{aligned}$$